```
# This Python 3 environment comes with many helpful analytics
libraries installed
# It is defined by the kaggle/python Docker image:
https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/"
directory
# For example, running this (by clicking run or pressing Shift+Enter)
will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/)
that gets preserved as output when you create a version using "Save &
Run All"
# You can also write temporary files to /kaggle/temp/, but they won't
be saved outside of the current session
```

Youtube Video Link : https://youtu.be/Asm2_qgBK48

```
"""from google.colab import drive
drive.mount('/content/drive')"""

{"type":"string"}

import warnings
warnings.filterwarnings('ignore')

data = pd.read_csv('/content/IMDB-Movie-Data.csv')
```

1. Display Top 10 Rows of The Dataset

This project analyzes an IMDb movie dataset using Python. The goal is to explore movie ratings, genres, and trends using data analysis and visualization techniques. Additional analysis has been performed to extract meaningful insights from the dataset.

```
data.head(10)
```

{"summary":"{\n \"name\": \"data\",\n \"rows\": 1000,\n \"fields\": [\n {\n \"column\": \"Rank\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 288,\n \"min\": 1,\n \"max\": 1000,\n \"num_unique_values\": 1000,\n \"samples\": [\n 522,\n 738,\n 741\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Title\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 999,\n \"samples\": [\n \"Olympus Has Fallen\",\n \"Man on a Ledge\",\n \"The Girl with All the Gifts\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Genre\",\n \"properties\": {\n \"dtype\": \"category\",\n \"num_unique_values\": 207,\n \"samples\": [\n \"Mystery,Romance,Sci-Fi\",\n \"Drama,Mystery,Sci-Fi\",\n \"Drama,Mystery,Romance\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Description\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 1000,\n \"samples\": [\n \"A lawyer finds himself in over his head when he gets involved in drug trafficking.\",\n \"A CIA agent on the ground in Jordan hunts down a powerful terrorist leader while being caught between the unclear intentions of his American supervisors and Jordan Intelligence.\",\n \"A titan of industry is sent to prison after she's caught insider trading. When she emerges ready to rebrand herself as America's latest sweetheart, not everyone she screwed over is so quick to forgive and forget.\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Director\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 644,\n \"samples\": [\n \"Patricia Riggen\",\n \"Gregory Wilson\",\n \"Chris McCoy\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Actors\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 996,\n \"samples\": [\n \"Adrian Titieni, Maria-Victoria Dragus, Lia Bugnar,Malina Manovici\",\n \"Madina Nalwanga, David Oyelowo, Lupita Nyong'o, Martin Kabanza\",\n \"Ry\\u00fbnosuke Kamiki, Mone Kamishiraishi, Ry\\u00f4 Narita, Aoi Yuki\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Year\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 3,\n \"min\": 2006,\n \"max\": 2016,\n \"num_unique_values\": 11,\n \"samples\": [\n 2011,\n 2014,\n 2010\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Runtime (Minutes)\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 18,\n \"min\": 66,\n \"max\": 191,\n \"num_unique_values\": 94,\n \"samples\": [\n 106,\n 99,\n 146\

n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Rating\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 0.9454287892779637,\n   \"min\": 1.9,\n   \"max\":
9.0,\n   \"num_unique_values\": 55,\n   \"samples\": [\n
7.4,\n   6.1,\n   4.1\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"Votes\",\n   \"properties\": {\
n   \"dtype\": \"number\",\n   \"std\": 188762,\n
\"min\": 61,\n   \"max\": 1791916,\n
\"num_unique_values\": 997,\n   \"samples\": [\n
214994,\n   4370,\n   23713\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"Revenue (Millions)\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
103.25354047492473,\n   \"min\": 0.0,\n   \"max\": 936.63,\n
\"num_unique_values\": 814,\n   \"samples\": [\n   89.02,\
n   23.23,\n   202.85\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"Metascore\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
17.194757023263836,\n   \"min\": 11.0,\n   \"max\": 100.0,\n
\"num_unique_values\": 84,\n   \"samples\": [\n   27.0,\n
76.0,\n   47.0\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   }\n]\
n}","type":"dataframe","variable_name":"data"}

2. Check Last 10 Rows of The Dataset

```
data.tail(10)
```

{"summary":"{\n \"name\": \"data\",\n \"rows\": 10,\n \"fields\":
[\n   {\n   \"column\": \"Rank\",\n   \"properties\": {\n
\"dtype\": \"number\",\n   \"std\": 3,\n   \"min\": 991,\n
\"max\": 1000,\n   \"num_unique_values\": 10,\n
\"samples\": [\n   999,\n   992,\n   996\n
],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   },\n   {\n   \"column\": \"Title\",\n   \"properties\":
{\n   \"dtype\": \"string\",\n   \"num_unique_values\": 10,\
n   \"samples\": [\n   \"Search Party\",\n
\"Taare Zameen Par\",\n   \"Secret in Their Eyes\"\n   ],\
n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   },\n   {\n   \"column\": \"Genre\",\n   \"properties\":
{\n   \"dtype\": \"string\",\n   \"num_unique_values\": 10,\
n   \"samples\": [\n   \"Adventure,Comedy\",\n
\"Drama,Family,Music\",\n   \"Crime,Drama,Mystery\"\
n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":

\"Description\",\n   \"properties\": {\n   \"dtype\": \"string\",\n   \"num_unique_values\": 10,\n   \"samples\": [\n   \"A pair of friends embark on a mission to reunite their pal with the woman he was going to marry.\",\n   \"An eight-year-old boy is thought to be a lazy trouble-maker, until the new art teacher has the patience and compassion to discover the real problem behind his struggles in school.\",\n   \"A tight-knit team of rising investigators, along with their supervisor, is suddenly torn apart when they discover that one of their own teenage daughters has been brutally murdered.\"\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Director\",\n   \"properties\": {\n   \"dtype\": \"string\",\n   \"num_unique_values\": 10,\n   \"samples\": [\n   \"Scot Armstrong\",\n   \"Aamir Khan\",\n   \"Billy Ray\"\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Actors\",\n   \"properties\": {\n   \"dtype\": \"string\",\n   \"num_unique_values\": 10,\n   \"samples\": [\n   \"Adam Pally, T.J. Miller, Thomas Middleditch,Shannon Woodward\",\n   \"Darsheel Safary, Aamir Khan, Tanay Chheda, Sachet Engineer\",\n   \"Chiwetel Ejiofor, Nicole Kidman, Julia Roberts, Dean Norris\"\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Year\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 3,\n   \"min\": 2007,\n   \"max\": 2016,\n   \"num_unique_values\": 9,\n   \"samples\": [\n   2014,\n   2007,\n   2015\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Runtime (Minutes)\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 23,\n   \"min\": 87,\n   \"max\": 165,\n   \"num_unique_values\": 9,\n   \"samples\": [\n   93,\n   165,\n   94\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Rating\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0.9065195959149355,\n   \"min\": 5.3,\n   \"max\": 8.5,\n   \"num_unique_values\": 9,\n   \"samples\": [\n   5.6,\n   8.5,\n   6.2\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Votes\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 55746,\n   \"min\": 4881,\n   \"max\": 164088,\n   \"num_unique_values\": 10,\n   \"samples\": [\n   4881,\n   102697,\n   27585\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Revenue (Millions)\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 24.21936297380967,\n   \"min\": 1.2,\n   \"max\": 60.13,\n   \"num_unique_values\": 8,\n   \"samples\": [\n   1.2,\n   17.54,\n   45.8\n   ],\n   \"semantic_type\": \"\",\n

```
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Metascore\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 13.200378782444085,\n        \"min\":
11.0,\n        \"max\": 50.0,\n        \"num_unique_values\": 9,\n
\"samples\": [\n          22.0,\n          42.0,\n          46.0\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    }\n  ]\n}","type":"dataframe"}
```


3. Find Shape of Our Dataset (Number of Rows And Number of Columns)

```
data.shape
```

```
(1000, 12)
```

```
print('Number of Rows',data.shape[0])
print('Number of Columns',data.shape[1])
```

```
Number of Rows 1000
Number of Columns 12
```

4. Getting Information About Our Dataset Like Total Number Rows, Total Number of Columns, Datatypes of Each Column And Memory Requirement

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 12 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   Rank                1000 non-null   int64
 1   Title               1000 non-null   object
 2   Genre               1000 non-null   object
 3   Description         1000 non-null   object
 4   Director            1000 non-null   object
 5   Actors              1000 non-null   object
 6   Year                1000 non-null   int64
 7   Runtime (Minutes)   1000 non-null   int64
 8   Rating              1000 non-null   float64
```

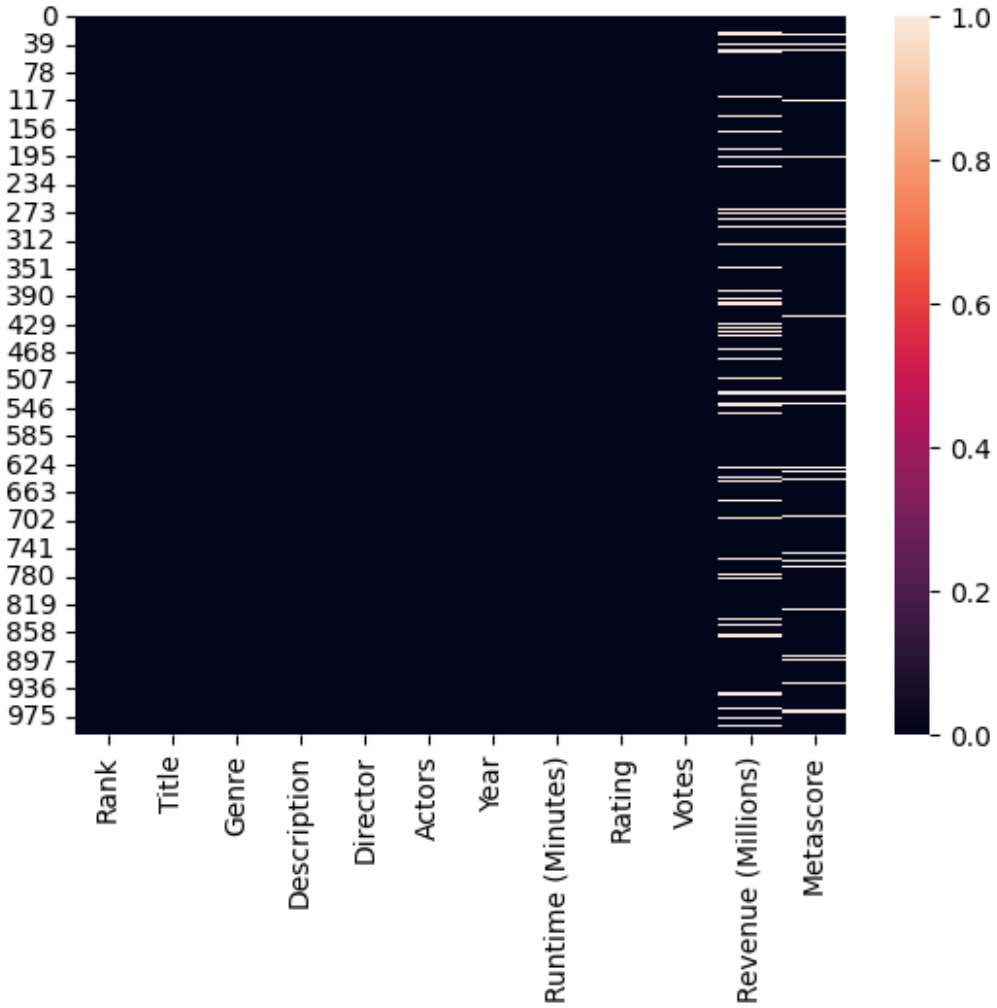
```
 9   Votes                1000 non-null   int64
 10  Revenue (Millions)  872 non-null    float64
 11  Metascore           936 non-null    float64
dtypes: float64(3), int64(4), object(5)
memory usage: 93.9+ KB
```

5. Check Null Values In The Dataset

```
data.isnull().sum()
```

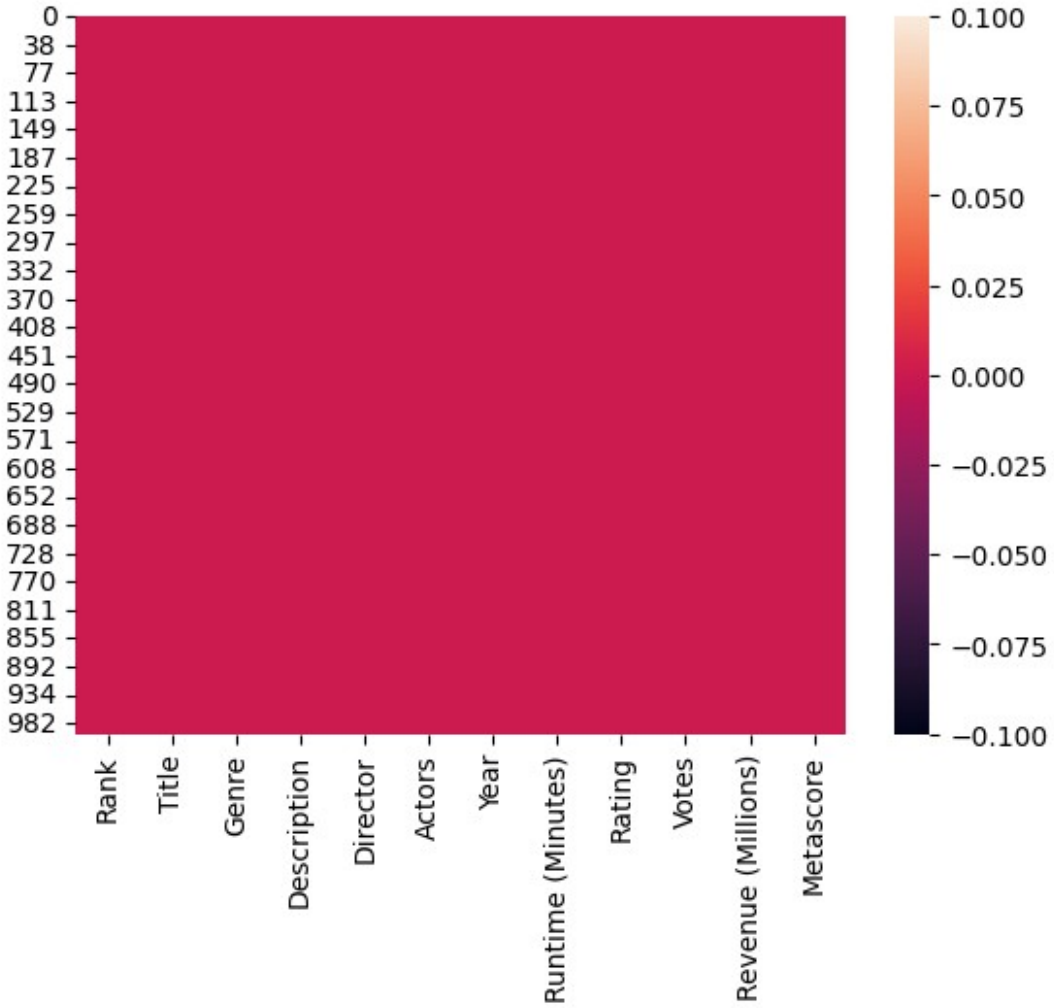
```
Rank                     0
Title                    0
Genre                    0
Description              0
Director                 0
Actors                   0
Year                     0
Runtime (Minutes)        0
Rating                   0
Votes                    0
Revenue (Millions)     128
Metascore               64
dtype: int64
```

```
import matplotlib.pyplot as plt
import seaborn as sns
sns.heatmap(data.isnull())
plt.show()
```



6. Drop All The Missing Values

```
data = data.dropna(axis=0)

sns.heatmap(data.isnull())
plt.show()
```



7. Check For Duplicate Data

```
dup_data=data.duplicated().any()
print("Are there any duplicated values in data?",dup_data)
```

```
Are there any duplicated values in data? False
```

8. Get Overall Statistics About The DataFrame

```
data.describe()
```


```
{"summary":"{\n  \"name\": \"data\",\n  \"rows\": 8,\n  \"fields\": [\n    {\n      \"column\": \"Rank\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 333.92550255275137,\n        \"min\": 1.0,\n        \"max\": 1000.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n
```

485.2470167064439,\n   475.5,\n   838.0\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"Year\",\n   \"properties\": {\n
\"dtype\": \"number\",\n   \"std\": 769.7713287499067,\n
\"min\": 3.172359915266028,\n   \"max\": 2016.0,\n
\"num_unique_values\": 8,\n   \"samples\": [\n
2012.5071599045345,\n   2013.0,\n   838.0\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"Runtime (Minutes)\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
264.1807671963779,\n   \"min\": 18.470922051554556,\n
\"max\": 838.0,\n   \"num_unique_values\": 8,\n
\"samples\": [\n   114.63842482100239,\n   112.0,\n
838.0\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Rating\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 294.3064953884777,\n   \"min\": 0.8777538418027501,\n
\"max\": 838.0,\n   \"num_unique_values\": 8,\n
\"samples\": [\n   6.814319809069212,\n   6.9,\n
838.0\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Votes\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 598181.3436789116,\n   \"min\": 178.0,\n   \"max\":
1791916.0,\n   \"num_unique_values\": 8,\n   \"samples\": [\
n   193230.25178997614,\n   136879.5,\n   838.0\n
],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   },\n   {\n   \"column\": \"Revenue (Millions)\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
385.4296330380251,\n   \"min\": 0.0,\n   \"max\": 936.63,\n
\"num_unique_values\": 8,\n   \"samples\": [\n
84.5645584725537,\n   48.150000000000006,\n   838.0\n
],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   },\n   {\n   \"column\": \"Metascore\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
279.24227712556353,\n   \"min\": 11.0,\n   \"max\": 838.0,\n
\"num_unique_values\": 8,\n   \"samples\": [\n
59.575178997613364,\n   60.0,\n   838.0\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   }\n]\n}","type":"dataframe"}

9. Display Title of The Movie Having Runtime >= 180 Minutes

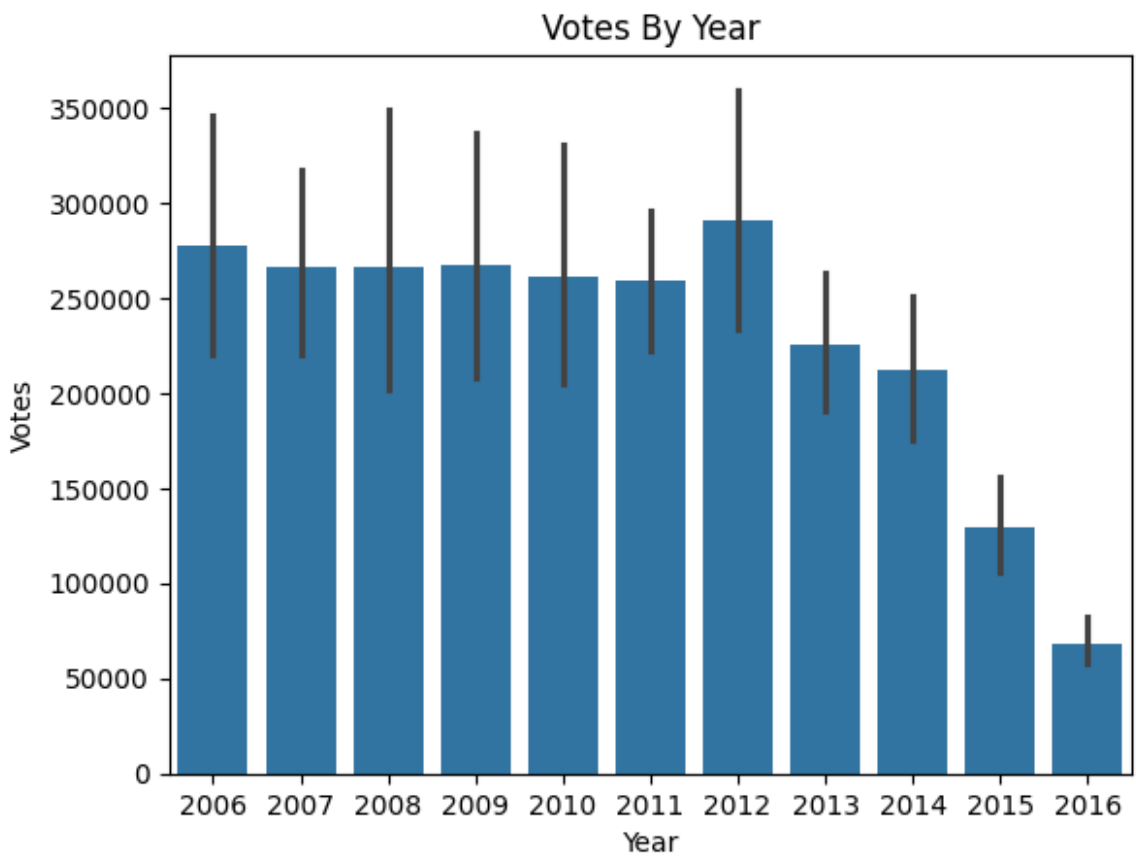
```
data[data['Runtime (Minutes)']>=180]['Title']
```

```
82     The Wolf of Wall Street
88           The Hateful Eight
```

```
311              La vie d'Adèle
Name: Title, dtype: object
```

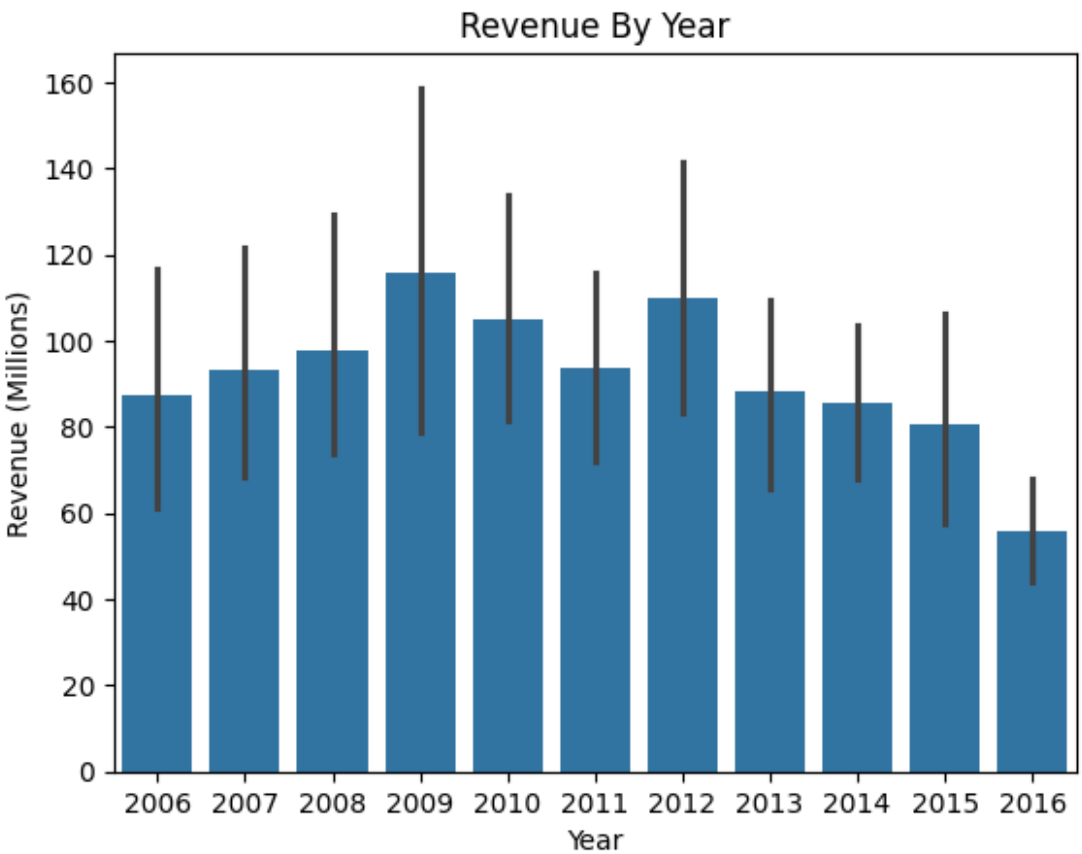
10. In Which Year There Was The Highest Voting?

```
sns.barplot(x='Year',y='Votes',data=data)
plt.title("Votes By Year")
plt.show()
```



11. In Which Year There Was The Highest Revenue?

```
sns.barplot(x='Year',y='Revenue (Millions)',data=data)
plt.title("Revenue By Year")
plt.show()
```



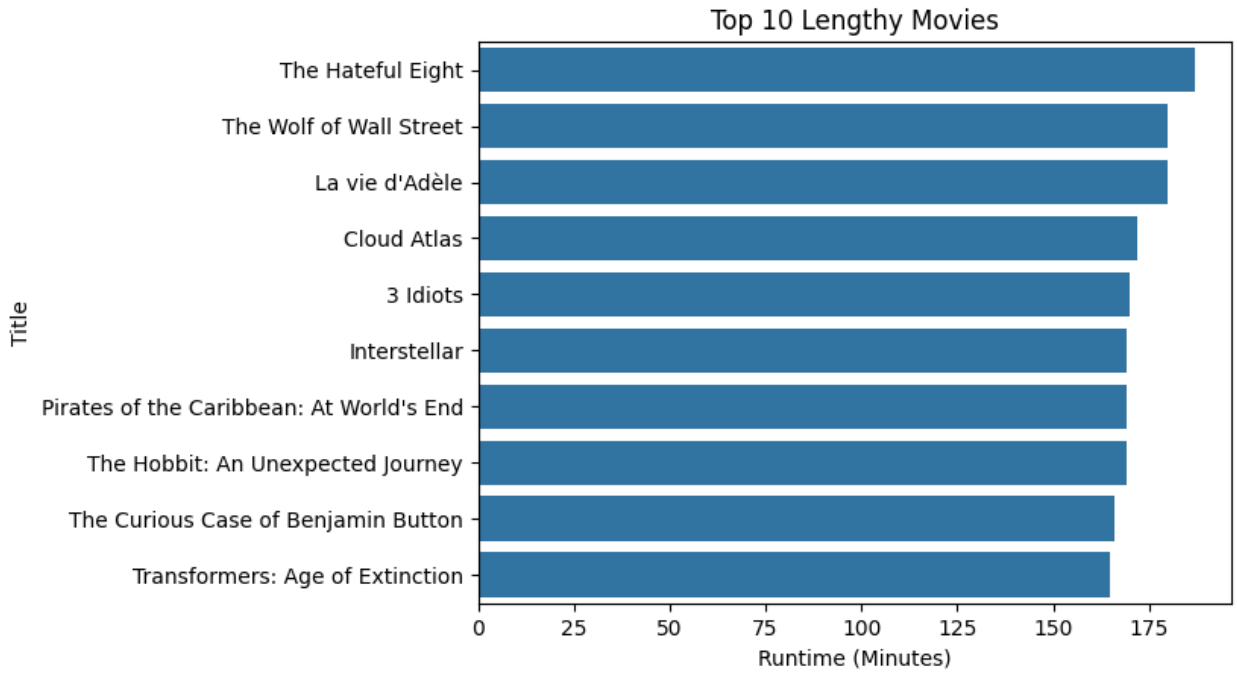
12. Find The Average Rating For Each Director

```
data.groupby('Director')['Rating'].mean().sort_values(ascending=False)
```

```
Director
Christopher Nolan                    8.68
Makoto Shinkai                       8.60
Olivier Nakache                      8.60
Aamir Khan                           8.50
Florian Henckel von Donnersmarck     8.50
                                     ...
Sam Taylor-Johnson                   4.10
Joey Curtis                          4.00
George Nolfi                         3.90
James Wong                           2.70
Jason Friedberg                      1.90
Name: Rating, Length: 524, dtype: float64
```

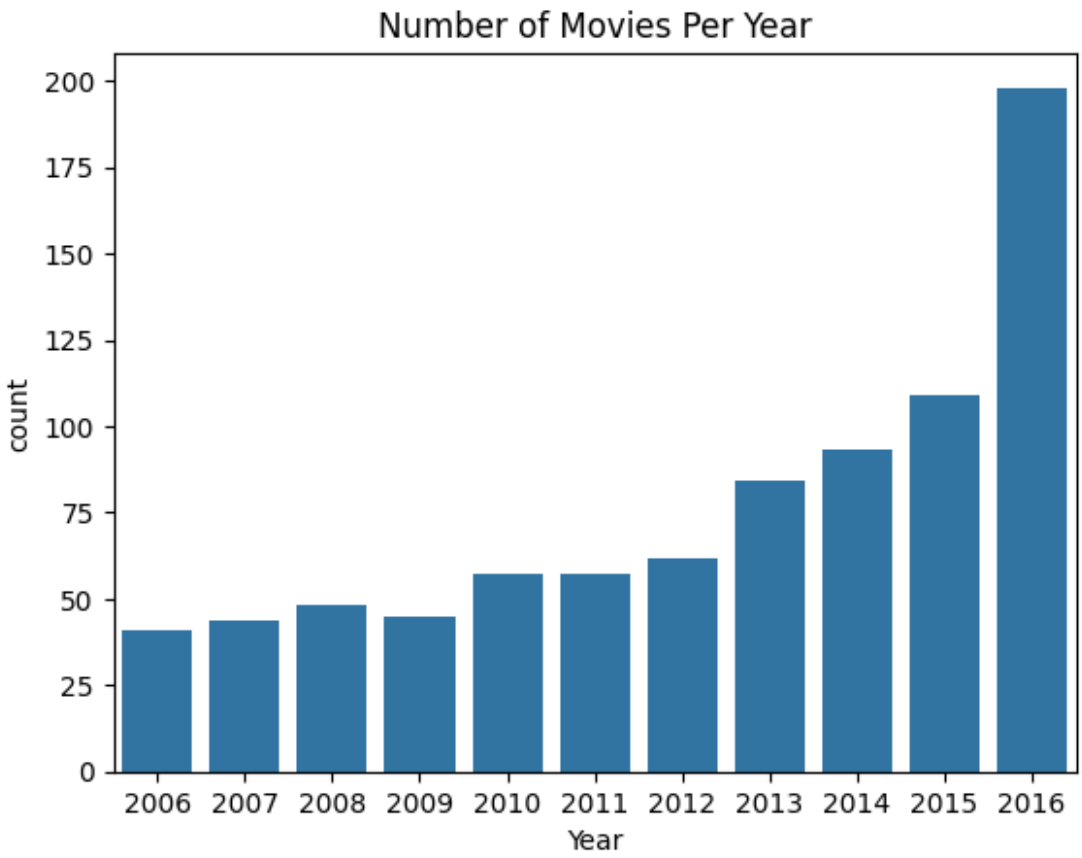
13. Display Top 10 Lengthy Movies Title

```
le =data.nlargest(10,'Runtime (Minutes)')[['Title','Runtime
(Minutes)']]. \
set_index('Title')

sns.barplot(x=le['Runtime (Minutes)'], y=le.index)
plt.title('Top 10 Lengthy Movies')
plt.show()
```



14. Display Number of Movies Per Year

```
sns.countplot(x='Year',data=data)
plt.title("Number of Movies Per Year")

Text(0.5, 1.0, 'Number of Movies Per Year')
```



15. Find Most Popular Movie Title (Higest Revenue)

```
data.columns
```

```
Index(['Rank', 'Title', 'Genre', 'Description', 'Director', 'Actors', 'Year',
       'Runtime (Minutes)', 'Rating', 'Votes', 'Revenue (Millions)',
       'Metascore'],
      dtype='object')
```

```
data[data['Revenue (Millions)'].max() == data['Revenue (Millions)']]['Title']
```

```
50    Star Wars: Episode VII - The Force Awakens
Name: Title, dtype: object
```

16. Display Top 10 Highest Rated Movie Titles And its Directors

```
top_10=data.nlargest(10,'Rating')
[['Title','Rating','Director']].set_index('Title')

top_10
```


```
{"summary":"{\n  \"name\": \"top_10\",\n  \"rows\": 10,\n  \"fields\": [\n  {\n  \"column\": \"Title\",\n  \"properties\": {\n  \"dtype\": \"string\",\n  \"num_unique_values\": 10,\n  \"samples\": [\n  \"Whiplash\",\n  \"Inception\",\n  \"The Prestige\"\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Rating\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 0.1663329993316621,\n  \"min\": 8.5,\n  \"max\": 9.0,\n  \"num_unique_values\": 4,\n  \"samples\": [\n  8.8,\n  8.5,\n  9.0\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Director\",\n  \"properties\": {\n  \"dtype\": \"string\",\n  \"num_unique_values\": 6,\n  \"samples\": [\n  \"Christopher Nolan\",\n  \"Makoto Shinkai\",\n  \"Florian Henckel von Donnersmarck\"\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  }\n  ]\n}","type":"dataframe","variable_name":"top_10"}
```


```
sns.barplot(top_10['Rating'],top_10.index)
plt.title("Display Top 10 Highest Rated Movie Titles")
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipython-input-1471653688.py in <cell line: 0>()
----> 1 sns.barplot(top_10['Rating'],top_10.index)
      2 plt.title("Display Top 10 Highest Rated Movie Titles")

TypeError: barplot() takes from 0 to 1 positional arguments but 2 were given
```

17. Display Top 10 Highest Revenue Movie Titles

```
data.columns

data.sort_values(by='Revenue (Millions)',ascending=False).head(10)
```

```
top_10 = data.nlargest(10,'Revenue (Millions)')
[['Title','Director','Revenue (Millions)']].set_index('Title')

sns.barplot(top_10['Revenue (Millions)'],top_10.index)
plt.title("Display Top 10 Highest Revenue Movie Titles")
plt.show()
```

18. Find Average Rating of Movies Year-wise

```
data.columns

data1=data.groupby('Year')['Year','Rating'].mean().\
sort_values(by='Rating',ascending=False).set_index('Year')

data1

plt.figure(figsize=(10,5))
sns.barplot(data1.index,data1['Rating'])
plt.show()
```

19. Does Rating Affect The Revenue?

```
sns.scatterplot(x='Rating',y='Revenue (Millions)',data=data)
```

Answer : Yes

20. Classify Movies Based on Ratings [Good,Better and Best]

```
data.columns

def rating(rating):
    if rating>=7.0:
        return 'Excellent'
    elif rating>=6.0:
        return 'Good'
    else:
        return 'Average'

data['rating_cat']=data['Rating'].apply(rating)

data.head(1)
```

21. Count Number of Action Movies

```
list1=[]
for value in data['Genre']:
    list1.append(value.split(','))

data['temp']=list1

genre=input("Enter Genre you want to count : ").title()
count=0
for value in data['temp']:
    if genre in value:
        count=count+1
print("Total Count is",count)
```

OR

```
len(data[data['Genre'].str.contains('action',case=False)])
```

From the analysis, we observe patterns in ratings across genres and years. The dataset reveals how audience preferences vary and highlights top-performing films. This project demonstrates how Python can be used effectively for real-world data exploration.